

Refleksjonsnotat – Mattehjelpen – Gruppe 2

Når er appen et hjelpemiddel og når er den juks? Trekk paralleller til kalkulatoren.

Vi mener at MatteHjelpen kan være et nyttig hjelpemiddel når brukeren fortsatt gjør det faglige arbeidet selv. Det vil si at studenten forstår hva oppgaven spør om, kjenner relevante formler og regler, velger en metode og kan vurdere om svaret virker rimelig. Appen kan da brukes omtrent som en kalkulator: Den kan gjøre rutineberegninger, forklare mellomregning og hjelpe brukeren videre dersom vedkommende står fast.

Forskjellen er at en KI-app kan gjøre mer enn en kalkulator. Den kan foreslå metode, formulere forklaringer og tolke tekst. Det gjør den nyttig, men også mer risikabel. Den kan gi et overbevisende svar selv om den har misforstått oppgaven eller valgt feil metode. Derfor bør brukeren ikke bare godta svaret, men bruke appen til å lære og kontrollere egen framgangsmåte.

Vi mener grensen mot juks går når appen overtar tenkearbeidet. Dersom en student får en ferdig løsning og leverer den uten å forstå eller kunne forklare framgangsmåten, har studenten ikke selv vist kompetansen oppgaven skal måle. Det er forskjell på å bruke appen til å få forklart hvorfor et steg gjøres, og å bruke den til å skrive hele besvarelsen.

Testene våre viser også hvorfor appen bør brukes som støtte, ikke som fasit. I Google-kjøringen fikk vi riktige og validerte svar på de fire første oppgavene med tools aktivert. Uten tools svarte appen derimot «ingen løsning» på de samme oppgavene, og ingen av svarene ble validert. Dette viser at verktøyene kan gjøre appen mye bedre, men at brukeren fortsatt må forstå hva appen faktisk har kontrollert.

Hva må du kunne for å oppdage at appen tar feil?

For å oppdage feil må brukeren forstå nok matematikk til å vurdere om metode, fortegn, enheter og størrelsesorden virker riktige. Det er ikke nødvendig å kunne utføre alle beregninger raskt i hodet, men man må kunne spørre: Passer dette svaret til oppgaven? Er svaret positivt eller negativt som forventet? Har svaret riktige enheter? Er tallet realistisk?

En viktig kontroll er å sette svaret inn i den opprinnelige likningen igjen. Dersom appen for eksempel foreslår $x = 8/3$ som løsning, kan vi kontrollere om verdien faktisk gjør likningen riktig. Når appen viser «validert: ja», er dette mer troverdig enn et svar som bare er formulert av KI-en. Likevel betyr ikke valideringen at hele løsningen nødvendigvis er riktig i en større sammenheng.

SymPy-valideringen kan kontrollere avgrensede, maskinlesbare matematiske påstander. Den kan for eksempel kontrollere en kandidat i en likning eller undersøke et konkret beregningsresultat. Den kan ikke avgjøre om oppgaven ble forstått riktig, om brukeren skrev inn riktige opplysninger, om nødvendige forutsetninger mangler, eller om svaret er forsvarlig

i en praktisk ingeniørsituasjon. Den kan heller ikke automatisk bevise en tekstlig argumentasjon, noe vi også så i bevisoppgaven om Pytagoras' læresetning.

OpenRouter-kjøringen viser at tools ikke er en garanti for at alt blir riktig. Med tools fikk appen flere riktige og validerte svar på de første oppgavene, men den løste ikke alle oppgavene. Den strevde blant annet med likningssystemet og en differensiallikning. Den fikk også riktige svar på polarform og determinant, men disse ble ikke validert. Uten tools fikk appen noen riktige svar, men ingen av dem ble validert. Dette viser at vi må skille mellom et svar som tilfeldigvis er riktig, og et svar som faktisk er kontrollert.

For å redusere feil må brukeren også skrive tydelige spørsmål. En uklar oppgave kan føre til at KI-en regner riktig på feil problem. Derfor bør vi oppgi variabler, enheter, forutsetninger og hva vi ønsker å finne. Tydelige spørsmål kan være forskjellen mellom et relevant og et irrelevant svar, men brukeren må fortsatt kontrollere resultatet.

Ville du stolt på den for å dimensjonere en bro? Hvorfor/hvorfor ikke?

Vi ville ikke stolt på MatteHjelpen alene for å dimensjonere en bro. Et matematisk riktig svar er ikke nok i ingeniørfag. Ved dimensjonering må en også ta hensyn til materialer, laster, vind, snø, grunnforhold, utmattelse, sikkerhetsfaktorer, byggemetode og gjeldende standarder. Dersom appen mangler én viktig opplysning eller tolker oppgaven feil, kan en tilsynelatende riktig beregning bli farlig å bruke.

Appen kan likevel være et nyttig støtteverktøy for en ingeniør. Den kan hjelpe med rutineberegninger, forklare framgangsmåter, foreslå kontrollspørsmål og gjøre det lettere å oppdage en feil. Den kan også være nyttig når en skal komme i gang med et problem. Det endelige ansvaret må imidlertid ligge hos en kvalifisert fagperson som forstår og godkjenner beregningene.

Dersom et KI-forslag brukes i en konstruksjon og det oppstår en feil, mener vi at ansvaret fortsatt ligger hos ingeniøren som velger å bruke og godkjenne løsningen. KI-en kan ikke holdes faglig ansvarlig. Flere personer bør derfor kontrollere viktige beregninger, fordi ulike fagpersoner kan oppdage ulike feil og utfordre hverandres antakelser.

Testresultatene våre gjør at vi har begrenset, men ikke blind, tillit til appen. Vi har mest tillit når appen har brukt tools og viser at resultatet er validert. Vi har mindre tillit når appen svarer uten tools, når den ikke kan validere svaret, eller når den ikke løser oppgaven. Google-testen måtte også avsluttes etter fire oppgaver fordi gratisnøkkelen gikk tom for tokens. Derfor kan vi ikke trekke altfor bastante konklusjoner fra én enkelt kjøring.

Før en slik app kunne vært tilgjengelig for andre studenter på nettet, måtte den vært testet på langt flere oppgaver og gjennomgått av både utviklere og fagpersoner. Den måtte også håndtere feil input, feil tolkning av tekst, API-nøkler, personvern, kostnader og misbruk på en trygg måte.

Hva vil det si å være «flink i matte» når slike verktøy finnes?

Å være flink i matte handler ikke bare om å regne raskt. Når kalkulatorer og KI kan utføre rutineberegninger, blir det enda viktigere å forstå problemet, velge riktig metode og vurdere om svaret er realistisk. Det viktigste er å vite hva som skal beregnes, hvorfor det skal beregnes, og hvordan resultatet kan kontrolleres.

En person som bruker KI må kunne forklare hvilke formler og regler som brukes. Ved oppgaver som får konsekvenser for andre mennesker, for eksempel i bygg- og ingeniørfag, må fagpersonen kunne stå inne for resultatet. Det er ikke nok å si at «KI-en sa det». Ingeniøren må kunne oppdage feil tall, feil forutsetninger, feil enheter og feil tolkning av oppgaven.

Kritisk tenkning er derfor en viktig del av å være flink i matte. Man må kunne kontrollere svar, stille spørsmål ved antakelser og vite når en løsning ikke er godt nok dokumentert. KI kan være raskere enn mennesker på enkelte beregninger og ha tilgang til mange formler, men den har ikke automatisk faglig skjønn eller ansvar.

KI kan gjøre oss bedre i matematikk dersom vi bruker den aktivt til læring. Den kan forklare et vanskelig steg, gi flere eksempler og hjelpe oss med å kontrollere egen framgangsmåte. Dersom vi bare ber om et ferdig svar og kopierer det, lærer vi mindre. Etter vår vurdering betyr det å være flink i matte i dag å kunne bruke verktøy på en klok måte, men samtidig beholde evnen til å tenke selv, kontrollere svar og ta ansvar for egne faglige valg.

Til selve kodingen begynte vi med å bruke Gemini, men fant raskt ut at Claude hadde mye bedre samarbeid med VS Code så vi byttet over til Claude for oppgaven. For refleksjonsnotatet sendte vi inn spørsmålene sammen med svarene våre og hva vi mente for å se hvor bra den kunne formulere notatet.

Vi har testet MatteHjelpen med to modeller av ulik kvalitet: Modell A (Google Gemini, gemini-flash-latest) som den sterkere modellen, og Modell B (OpenRouter, liquid/lfm-2.5-2.6b:free) som en liten, svakere gratismodell – valgt for å gi tydelig kontrast i kvalitet, slik Del B ber om. Begge kjøres via samme OpenAI-kompatible grensesnitt i llm_client.py, og byttes ved å kommentere inn/ut de tre linjene i .env.

Planen var å kjøre alle 10 testoppgavene med begge modeller, både med og uten tool-calling (SymPy-verktøyene), til sammen minst 40 kjøring. I praksis er ikke alle 40 kombinasjonene gjennomført, siden gratiskvotene (~20 kall/dag for Gemini, dagsgrense for OpenRouter) og periodevis overbelastning hos Google (503-feil på Gemini-endeppet) satte en naturlig grense for hvor mange kjøring vi fikk gjort samme dag. Tabellen under viser hvilke kombinasjoner som faktisk er testet.

Formålet er å sammenligne nøyaktighet, ærlighet om usikkerhet, tokenforbruk/kostnad og robusthet mellom en sterk og en svak gratismodell.

Med gratis nøkkel fra Google AI Studio:

Oppgave	Med tools	Uten tools
1) Deriver $x^3 \cdot \sin(x)$	Riktig: Ja Validert: Ja Tokens: 4892	Riktig: Ingen løsning Validert: Nei Tokens: 1521
2) Deriver $(2x+1)/(x^2+3)$	Riktig: Ja Validert: Ja Tokens: 5070	Riktig: Ingen løsning Validert: Nei Tokens: 1492
3) Integrer $x^2 \cdot \cos(x)$	Riktig: Ja Validert: Ja Tokens: 5504	Riktig: Ingen løsning Validert: Nei Tokens: 1453
4) Løs ligningen $2x^2 - 5x - 3 = 0$	Riktig: Ja Validert: Ja Tokens: 5242	Riktig: Ingen løsning Validert: Nei Tokens:
5) Løs likningssystemet $2x+y=3$, $x+3y=5$	Riktig: Validert: Tokens:	Riktig: Validert: Tokens:
6) Løs differensialligningen $y' + 4y = 0$	Riktig: Validert: Tokens:	Riktig: Validert: Tokens:
7) Løs $y'' + y = 0$	Riktig: Validert: Tokens:	Riktig: Validert: Tokens:
8) Finn polarform til $1 + i$	Riktig: Validert: Tokens:	Riktig: Validert: Tokens:

9) Finn determinanten til matrisen $\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$	Riktig: Validert: Tokens:	Riktig: Validert: Tokens:
10) Bevis Pytagoras' læresetning (bevisoppgave — ingen SymPy-svar mulig)	Riktig: Validert: Tokens:	Riktig: Validert: Tokens:

Med gratis nøkkel fra OpenRouter.ai:

Oppgave	Med tools	Uten tools
1) Deriver $x^3 \cdot \sin(x)$	Riktig: Ja Validert: Ja Tokens: 5339	Riktig: Nei Validert: Nei Tokens: 1546
2) Deriver $(2x+1)/(x^2+3)$	Riktig: Ja Validert: Ja Tokens: 5090	Riktig: Nei Validert: Nei Tokens: 1973
3) Integrer $x^2 \cdot \cos(x)$	Riktig: Ja Validert: Ja Tokens: 5169	Riktig: Ja Validert: Nei Tokens: 2981
4) Løs ligningen $2x^2 - 5x - 3 = 0$	Riktig: «Ja» Validert: Ja Tokens: 15777	Riktig: Nei Validert: Nei Tokens: 1647
5) Løs likningssystemet $2x+y=3, x+3y=5$	Riktig: Løste ikke oppgaven Validert: Nei Tokens: 0	Riktig: Ja Validert: Nei Tokens: 4473
6) Løs differensialligningen $y' + 4y = 0$	Riktig: Løste ikke oppgaven Validert: Nei Tokens: 17916	Riktig: Validert: Tokens:
7) Løs $y'' + y = 0$	Riktig: Validert: Tokens:	Riktig: Ja Validert: Nei Tokens: 9597
8) Finn polarform til $1 + i$	Riktig: Ja Validert: Nei Tokens: 5971	Riktig: Ja Validert: Nei Tokens: 2541
9) Finn determinanten til matrisen $\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$	Riktig: Ja Validert: Nei Tokens: 7536	Riktig: Ja Validert: Nei Tokens: 1817
10) Bevis Pytagoras' læresetning (bevisoppgave — ingen SymPy-svar mulig)	Riktig: Nei Validert: Nei Tokens: 5066	Riktig: Nei Validert: Nei Tokens: 4109

Applikasjonen («MatteHjelpen») ble testet ved hjelp av gratis nøkler/modeller fra både Google via Google AI Studio og OpenRouter.ai (liquid/lfm-2.5-2.6b:free), koblet opp mot SymPy som verktøy (tool) for matematisk verifikasjon. Hovedfunn under oppgaveløsningen:

- Avhengighet av verktøy (Tools): Modellen klarte å løse beregningsoppgavene når den brukte verktøyene korrekt, men mislyktes uten.
- Tredjepartsstabilitet (Error 503): En planlagt test av Pytagoras' bevis feilet tre dager på rad på grunn av *Error 503 (Service Unavailable)*. Siden gratislaget fra Google/OpenRouter har lav prioritet under høy trafikk, ble forespørslene avvist.
- Krasj ved feil parameternavn (Feilhåndtering): I en likningssett-oppgave krasjet koden fordi modellen kalte verktøyet med feil variabelnavn (matrice i stedet for matrise). Dette avslørte et behov for mer robust feilhåndtering i koden, slik at appen sender en feilmelding tilbake til modellen og lar den prøve på nytt, i stedet for å krasje helt.
- Hallusinererte verktøykall (Fake JSON): Modellen «lot som» den brukte et verktøy ved å skrive ut fiktiv JSON-kode direkte i svarteksten uten å utføre et reelt API-kall. Appen oppdaget dette korrekt, stolte ikke på friteksten, og flagget svaret med validert: false.
- Uendelige sløyfer og høyt tokenforbruk:
 - Modellen feilet ved å generere samme løsning og verifikasjon 10 ganger på rad (diff.likning $y'' + y = 0$), noe som førte til et uforholdsmessig høyt tokenforbruk (ca. 9 600 tokens på én oppgave).
 - Dette viser at koden bør ha strengere grenser for *maks antall tool-kall* og *maksimalt tokenforbruk*.

2. Dypdykk: Bevisoppgaver og begrensninger (Pytagoras' læresetning)

Da appen ble testet på rent tekstlige og geometriske bevis (som Pytagoras), avdekket testen tre tydelige svakheter ved mindre LLM-er:

1. Manglende verktøyvalidering: Siden beviser krever resonnement og ikke ren beregning, kunne ikke SymPy benyttes. Appen flagget derfor helt korrekt. Svaret kunne ikke valideres.
2. Språkblanding og hallusinasjoner: Modellen begynte å rote med fagspråk og oppfant ord (som «saturett», «rett rett triangel» og «legum»), med en blanding av svensk, dansk og norsk.

3. Resonneringssløyer: Modellen sporet av, innrømmet at den ble forvirret, og prøvde på nytt 4–5 ganger i samme svar uten å fullføre beviset. Dette førte til over 5 000 brukte tokens uten et gyldig resultat.

Appen er svært pålitelig når SymPy kan verifisere svaret (derivasjon, integrasjon osv.). Når oppgaven krever rent resonnement uten verktøystøtte, øker faren for hallusinasjoner drastisk. Det er derfor avgjørende at brukergrensesnittet tydelig advarer brukeren når et svar ikke er bekreftet av et verktøy.

3. Økonomisk analyse og skalerbarhet (1 000 studenter)

Basert på testene brukte modellen i snitt 7 000 tokens per oppgave (på grunn av re-try-sløyer og forklaringer). Med en estimert API-pris på ca. \$0.000001 per token via OpenRouter, ser kostnadsbildet slik ut ved skalert bruk:

- 1000 studenter × 10 oppgaver (10 000 oppgaver):
 - Totalt antall tokens = $10\,000 \times 7\,000 = 70\,000\,000$ tokens
 - Estimert kostnad = $70\,000\,000 \times \$0,000001 = \$70 \approx 658$ kr
- 1000 studenter × 50 oppgaver (50 000 oppgaver):
 - Totalt antall tokens = $50\,000 \times 7\,000 = 350\,000\,000$ tokens
 - Estimert kostnad = $350\,000\,000 \times \$0,000001 = \$350 \approx 3\,290$ kr

Selv om tokenprisen per oppgave er lav, vil feilsløyer og ubegrensede tool-kall mangedoble tokenforbruket. Å legge inn *caching* på vanlige oppgaver og å sette tak på maks antall tokens per forespørsel vil være helt nødvendig i en produksjonssammenheng.